Domain: Computer Science

Project Report: Discrete Event Systems Modeling

Specialty: Computer and Network Systems (M1CNS)

Academic Year: 2025-2026

Projet: Modélisation des systèmes à évènements discrets

Supervised by: Mr. Jeremy SOBIERAJ

Realized by: John-Michael JENY JEYARAJ

Step 1: Comprehension and Operating Hypotheses

Resources and Limits

Let’s assume the restaurant’s owner wants to be able to serve 3 groups of customers at once, and decides he wants 3 Cashiers and 3 Cooks and 3 Dining Tables. The restaurant is a little popular so in the Waiting Room there’s enough seats for 5 groups of people, so they can wait while other customers are being served.

Arrivals and Impatience

Customers enter the restaurant only if the waiting room has space. If all 5 seats in the queue are full, there is a chance the customer gets impatient and leaves.

Eat-In or Takeout

Customers can choose “Takeout” and leave as soon as the meal is prepared. "Eat-In" is only possible if a Table and Condiments are available, otherwise, the customer can choose to not wait and go with "Takeout".

Staff Tasks

Cashiers handle orders, clean tables, and refill condiments. One cashier cannot do these at the same time. A cook must be available to prepare a meal and goes to get the ingredients themselves.

Inventory Rules

Cooking uses 1 Stock. Eating-in uses 1 Condiment. 1 Stock can be equivalent to 5 condiments. Restocking is manual so a staff member must stop serving customers to unload the truck or refill condiments.

Step 2: Identification of Network Elements

Places

Waiting Room Seats, Free Cashiers, Free Cooks, Orders Placed, Order Ready

Interested Customers, Queue, Cooking, Customers Eating, Customers Satisfied

Reorder Signal, Delivery Truck, Free Tables, Cleaning Table, Stock Condiments

Transitions

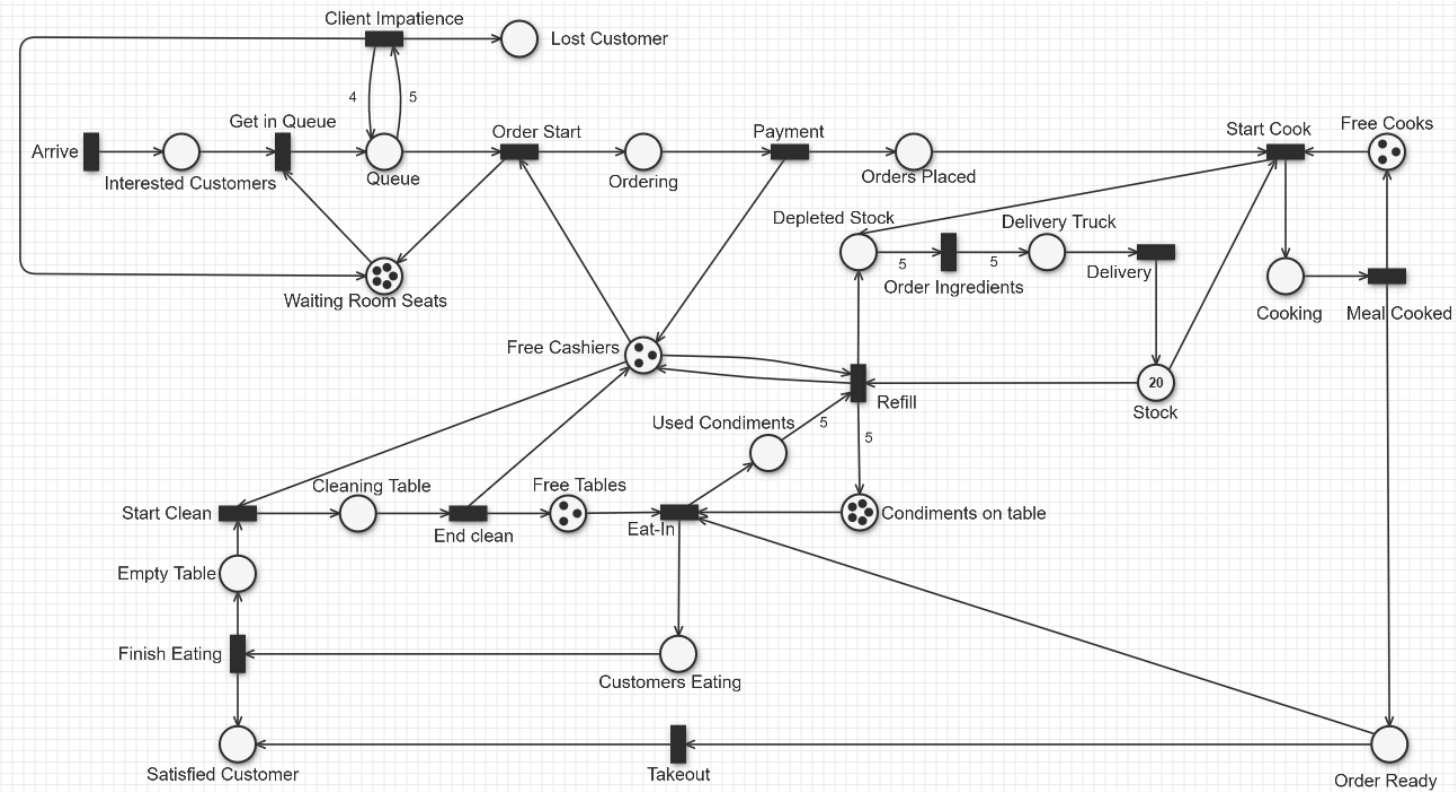
Arrive, Get in Queue, Client Impatience, Eat-In, Takeout, Finish Eating

Order Start, Payment, Start Cook, Meal Cooked

Start Clean, End Clean, Refill, Order From Supplier, Delivery

Step 3: Petri Net Construction

Model & Simulation

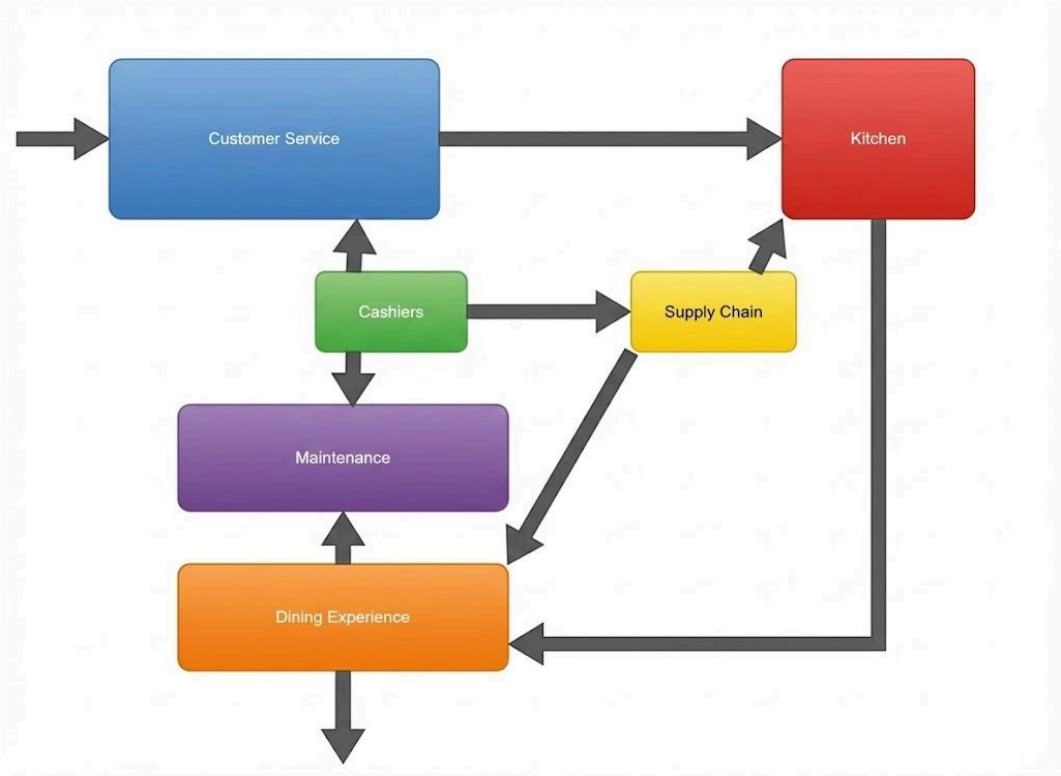


Weighted arcs implement specific rules: 2 for impatience, an output weight of 5 and input of 4 removes exactly one customer token only when the queue hits capacity. 2 for the supply chain, weights of 5 force batching, triggering a delivery only after 5 stock depletions. Lastly, 2 for the Refill transition uses weights to convert 1 bulk stock unit into 5 condiment portions.

3 Free Cashiers and 3 Free Cooks, so they are almost always busy and may cause conflicts between servicing customers and maintenance. 3 Free Tables is more than enough, and 5 Waiting Room Seats so the system is less sensitive to bursts of customers. Finally, 20 Stock and 5 Condiments so a failure to refill blocks the dining room.

Subnet Diagram

The subnet diagram helps visualize the restaurant's circular nature. It shows that the process isn't a straight line from arrival to departure, but that there is a continuous loop where the Dining Experience creates a burden on Maintenance, which must be resolved before the Customer Service cycle can restart.



It also highlights the central role of the Cashiers, who are positioned as the bridge connecting Customer Service to the support tasks (Maintenance and Supply Chain).

Step 4: Model Analysis

Resource Conservation

The network is stable because our resources are physically limited. The Queue can never exceed the number of Waiting Room Seats.

$$M(Waiting\ Room\ Seats) + M(Queue) = 5$$

Similarly, the human resources are conserved; the number of tokens representing Cashiers and Cooks remains constant and aren't lost. They circulate between being "Free" and performing tasks like Ordering, Cleaning, or Cooking, but they never disappear or multiply. The tables are also not lost once the Cashier is done with them.

$$M(FreeCashiers) + M(Ordering) + M(CleaningTable) + M(Refill) = 3$$

$$M(FreeCooks) + M(Cooking) = 3$$

$$M(FreeTables) + M(CustomersEating) + M(EmptyTable) + M(CleaningTable) = 3$$

The stock is constantly replenished when the trigger (Depleted Stock) is reached keeping a limit on the number of ingredients within the kitchen.

$$0 \leq M(Stock) \leq 20$$

Potential Deadlocks

There are no actual Deadlocks. The system is generally "live", with the small caveat that if Stock hits 0, cooking stops. We have modeled a Delivery Truck loop to replenish this, but it's quite slow. If Condiments on table or Free Tables run out, the "Eat-In" path is blocked, so without "Takeout," a customer would be stuck without a table to eat at.

Structural Boundedness

The inside of the restaurant is "bounded," meaning tokens don't pile up infinitely. The Queue is strictly limited to 5 groups because it is connected to the Waiting Room Seats. A customer cannot enter the Queue transition if there is no seat token available to grab. Also, the stock doesn't grow forever; the Delivery Truck is only called when we reach a certain amount of depleted stocks, so we never have too much food in the kitchen. Customers outside of the restaurant (Interested, Satisfied and Lost Customers) are not "bound".

Reachability Analysis

We verified that all the important states in the model can actually happen. The "Lost Customer" state is reachable, but only when the Queue gets crowded with 4 or more groups, triggering the impatience arc. The "Depleted Stock" place is also reachable, but it takes a while; it only happens after 5 Stock is used, which proves that our supply chain logic correctly waits for the inventory to drop before ordering more. Cashiers and Cooks can always reach their station after performing their task. And finally, once a Customer has placed their order they will always be able to reach Satisfied Customer.

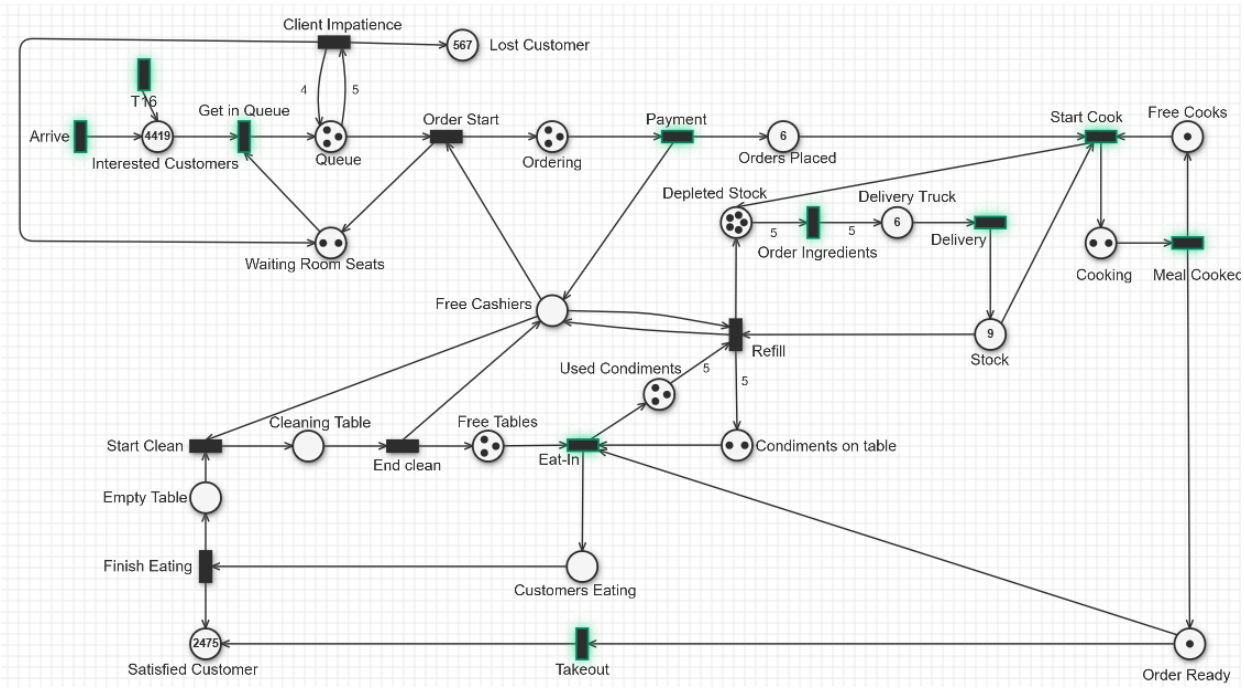
Step 5: Simulation and Interpretation

Scenario A: Normal Operation

With a slow arrival rate, the system works as intended. Waiting Room Seats remain available, so the "Impatience" loop is rarely triggered. Because there is less pressure on the counter, Free Cashiers are available often enough to fire the Refill and Start Clean transitions. This keeps the dining room functional for "Eat-In" customers. Cooks aren't overwhelmed by a high amount of orders, so 3 Cooks is more than enough. 20 stocks is too high of a number for this scenario; it's a resource that is extremely underused and could be reduced.

Scenario B: Saturated

We try to handle the saturation of customers at the very start of the model, by considering them as Interested Customers, because the restaurant is small but very popular. However, the Queue may sometimes rapidly reach its capacity of five tokens, keeping Waiting Room Seats at zero and physically blocking the Get in Queue transition. The high number of incoming customers would mean that the cashiers are busy taking orders and cleaning tables, so the queue bounces between 3 and 5 and triggering Client Impatience frequently, leading to losing customers. However, if the Cashiers are able to work efficiently through the high influx of customers, we instead see a small build up at Orders Placed, since we only have 3 Cooks. These are all the bottlenecks in the system, but normally under pressure the only bottleneck is the Queue size, which makes sure the restaurant isn't unrealistically packed. Despite only having 3 Free Tables, there usually isn't too much traffic after Order Ready, since customers who don't want to wait can just take their food as Takeout. 20 stock is close to the exact amount needed for the system to run smoothly without being slowed by the supply chain.



Conclusion

Working on this project was a fun experience in system design. My main focus was to create an elegant and simple model that accurately represents the restaurant while remaining visually readable, with a layout where no arrows cross over each other. The simulation proves that the bottleneck is the Queue size, which is actually a positive feature for the system. By limiting the number of people waiting in queue, it allows the restaurant to operate smoothly; although the staff can get a little overwhelmed when a big burst of customers enters at once, this bottleneck prevents an unrealistic number of customers being in the restaurant at once.